Spark Core, Spark SQL e Hive a confronto sul Flight Delay Dataset 2024

Corso di Big Data — Secondo progetto

Francesco Giovanardi
Università degli Studi Roma Tre — Prof. Riccardo Torlone

Anno accademico 2025/2026

Sommario

Sono state realizzate le analisi 3.1 (statistiche delle compagnie aeree, *Job 1*) e 3.2 (report dei ritardi per aeroporto e mese, *Job 2*) con Spark Core, Spark SQL e Hive. Il dataset è stato validato e ridotto a 9 colonne, poi usato in porzioni annidate e repliche da 2,1 MB a 1.027,3 MB. I job sono stati eseguiti in locale, su uno pseudo-cluster YARN e su AWS EMR, misurando tempi, stage e shuffle di ogni esecuzione; gli output delle tre tecnologie sono stati verificati riga per riga e coincidono. Codice: `https://github.com/fragiovis/BigData-FlightAnalysis`.

1 Scelte generali

- **Tecnologie**: Spark Core (RDD), Spark SQL (DataFrame e ottimizzatore Catalyst) e Hive (SQL tradotto in job MapReduce, che materializzano su disco i risultati intermedi): tre paradigmi diversi, dal funzionale a basso livello al dichiarativo.
- **Ambienti**: locale (`local[*]`), pseudo-cluster Hadoop/YARN a un nodo, cluster AWS EMR.
- **Output comune**: per ogni job le tre tecnologie scrivono lo stesso CSV (stesse colonne, ordinamento e arrotondamenti; liste separate da `|`; valori mancanti come campo vuoto). Questo permette di verificare automaticamente la correttezza (Sezione 6).

2 Preparazione dei dati

Il dataset Kaggle[1] è un CSV da 1,2 GB con 7.079.081 voli interni degli Stati Uniti e 35 colonne. La preparazione (`dataset/preprocessing.py`, PySpark) segue quattro fasi e registra l'esito di ciascuna in `results/qualita_dati.json`.

1. **Normalizzazione**: tipi espliciti, codici in maiuscolo e senza spazi, minuti per causa mancanti posti a 0 (nel dataset un valore mancante significa che alla causa non è attribuito ritardo).
2. **Validazione**: sette controlli; ogni riga è esclusa dal *primo* controllo che non supera, quindi è contata una sola volta, e i conteggi per motivo sono calcolati in un solo passaggio sui dati. Segue la rimozione dei duplicati sulla chiave del volo (data, compagnia, numero, tratta, orario programmato).
3. **Trasformazioni**, motivate sotto.
4. **Selezione** di 9 colonne: `month`, `op_unique_carrier`, `origin`, `dest`, `dep_delay`, `arr_delay`, `cancelled`, `cancellation_code`, `delay_code`. Otto sono usate dai job; `dest` (aeroporto di arrivo) non serve alle due analisi, basate sull'aeroporto di partenza, ma è mantenuta perché la traccia consente di calcolare il Job 1 anche per tratta. Il file scende a 205,5 MB.

[1] `https://www.kaggle.com/datasets/hrishitpatil/flight-data-2024`

Passo	Tipo	Descrizione	Righe
`dirottato`	esclusione	Volo dirottato: atterra in un aeroporto diverso da quello previsto, quindi ritardo in arrivo e cancellazione non sono confrontabili con gli altri voli	17.499
`campi_chiave_mancanti`	esclusione	Compagnia, aeroporto di partenza o di arrivo, mese o flag di cancellazione mancanti	0
`mese_non_valido`	esclusione	Mese fuori dall'intervallo 1–12	0
`codice_aeroporto_non_valido`	esclusione	Codice aeroporto diverso da tre lettere maiuscole (IATA)	0
`cancellazione_incoerente`	esclusione	Flag di cancellazione diverso da 0/1, oppure incoerente con la presenza del codice di cancellazione	0
`ritardi_mancanti`	esclusione	Volo non cancellato senza ritardo in partenza o in arrivo	0
`cause_incoerenti`	esclusione	Minuti attribuiti alle cause di ritardo la cui somma differisce dal ritardo in arrivo di oltre 1 minuto	0
`duplicati`	esclusione	Stesso volo ripetuto (data, compagnia, numero di volo, tratta e orario programmato)	0
`ritardo_cancellati`	trasformazione	Voli cancellati con un ritardo registrato (hanno lasciato il gate prima della cancellazione): ritardi azzerati, così non entrano nelle fasce di ritardo	3.345
`causa_prevalente`	trasformazione	Voli con minuti attribuiti alle cause di ritardo: delay_code = causa con più minuti	1.449.970

Tabella 1: Passi di validazione e trasformazione nell'ordine di applicazione, con le righe escluse o modificate.

Il dataset di origine è coerente: tutti i controlli sono superati e non ci sono duplicati. Vengono esclusi solo i 17.499 voli dirottati (0,25 %), che atterrano in un aeroporto diverso da quello previsto e hanno quindi un ritardo in arrivo non confrontabile; restano 7.061.582 voli. Le trasformazioni sono:

- **Voli cancellati senza ritardo**: 3.345 voli cancellati hanno un ritardo in partenza (hanno lasciato il gate prima della cancellazione). Un volo cancellato non è un volo in ritardo: i ritardi vengono rimossi, così il volo conta nel tasso di cancellazione e nelle cause ma non nelle fasce né nelle medie.
- **Causa prevalente**: il dataset fornisce i minuti di ritardo per cinque cause. Per i 1.449.970 voli che ne hanno, `delay_code` è la causa con più minuti, così la frequenza di una causa è il numero di voli di cui è la causa principale, confrontabile con il numero di cancellazioni per una causa.
- **Codici senza collisioni**: la lettera `C` indica "sistema aereo nazionale" tra le cause di cancellazione ma è la sigla naturale di "compagnia" tra quelle di ritardo. Per contarle insieme, come chiede la traccia, i codici sono resi espliciti: `CANC_*` per le cancellazioni, `DELAY_*` per i ritardi.

Sono stati mantenuti i 556 voli con più di 24 ore di ritardo (voli riprogrammati reali; la traccia chiede il ritardo massimo) e i 3 aeroporti con meno di 50 voli, che producono righe proprie senza alterare le altre.

3 Dataset di dimensione crescente

- **Porzioni annidate** (1%, 20%, 50%, 70%): a ogni volo viene assegnato una volta un numero casuale $u \in [0, 1)$ con seme fisso e la porzione del p% contiene i voli con $u < p$. Le porzioni sono riproducibili e annidate ($1\% \subset 20\% \subset \ldots \subset 100\%$, verificato): i tempi dipendono solo dal volume.
- **Repliche controllate** ($2\times$, $5\times$): cartelle HDFS con 2 e 5 copie del dataset completo, create con copie interne a HDFS. I risultati sono prevedibili (conteggi moltiplicati, medie e cause invariate), quindi verificabili.

Su HDFS ogni dataset è una cartella: Spark e Hive leggono la cartella intera e la tabella esterna di Hive vi punta direttamente, senza copie.

Dataset	Dimensione	Tipo	Righe	MB su HDFS
`flights_1`	1%	porzione annidata	71.085	2,1
`flights_20`	20%	porzione annidata	1.413.555	41,1
`flights_50`	50%	porzione annidata	3.531.974	102,8
`flights_70`	70%	porzione annidata	4.943.811	143,8
`flights_cleaned`	100%	completo	7.061.582	205,5
`flights_x2`	2×	replica (2 copie)	14.123.164	410,9
`flights_x5`	5×	replica (5 copie)	35.307.910	1.027,3

Tabella 2: Dataset usati negli esperimenti.

4 Ambienti e metodologia

- **Locale**: MacBook Air, Apple M1 (8 core), 8 GB di RAM; Java 21, Hadoop 3.4.1, Spark 3.5.5, Hive 4.0.0 (su MapReduce). Spark esegue tutto in una JVM con 8 thread; Hive esegue i job MapReduce nel processo (`mapreduce.framework.name=local`). I dati sono letti comunque da HDFS.
- **Pseudo-cluster YARN**, sulla stessa macchina: NodeManager con 4 GB e 4 vcore; Spark in modalità client con 2 executor da 1 core e 1 GB (valori predefiniti); Hive con container MapReduce da 1 GB. Le librerie di Spark sono su HDFS (`spark.yarn.jars`) e non vengono ricaricate a ogni job.
- **AWS EMR**: 1 nodo primario e 2 nodi core `m5.xlarge` (4 vCPU, 16 GB); dati da S3 a HDFS; su EMR Hive usa il motore Tez. Dataset dall'1% al 100%, un'esecuzione per combinazione, tempo totale.

Un modulo comune (`bench/runner.py`, usato da `benchmark.py` e dalla dashboard Streamlit del repository) lancia ogni job e registra: **tempo totale** (dall'avvio dello script alla fine, incluse JVM, sottomissione e scrittura), **tempo di calcolo** (per Spark la durata dell'applicazione dall'*event log*, per Hive la durata delle istruzioni riportata da beeline), il loro scarto (**overhead**) e le **metriche del motore** (job, stage, task e byte di shuffle per Spark; job MapReduce e byte letti da HDFS per Hive). In locale e su YARN sono stati eseguiti tutti i 7 dataset, 3 tecnologie e 2 job (42 e 42 esecuzioni); su EMR 30. Tabelle e grafici sono prodotti da `docs/relazione/genera_dati.py` a partire dai risultati registrati.

5 Job 1 — Statistiche delle compagnie aeree

Per ogni coppia (compagnia, aeroporto di partenza): numero di voli, ritardo minimo, massimo e medio in arrivo, tasso di cancellazione e mesi di attività. Si è scelto l'aeroporto di partenza (e non la tratta) per coerenza con il Job 2. L'output ha una riga per coppia, ordinata per compagnia e per numero di voli decrescente. I voli cancellati contano nei voli e nel tasso di cancellazione ma non hanno ritardo in arrivo, quindi sono esclusi da minimo, massimo e media (come fa `AVG` in SQL con i valori mancanti).

Spark Core. Ogni volo diventa una coppia chiave-accumulatore; `reduceByKey` combina gli accumulatori prima in ogni partizione e poi dopo lo shuffle, per cui viaggia un solo accumulatore per chiave e partizione.

```
dati   ← textFile(input) senza intestazione        // colonne individuate per nome dall'intestazione
coppie ← dati.map(r → ((r.compagnia, r.origine), (1, arr_valido, arr, arr, arr_o_0, r.cancellato, {r.mese})))
stat   ← coppie.reduceByKey(somma voli e validi, min/max ignorando i valori mancanti,
                            somma ritardi e cancellati, unione dei mesi)
output ← stat.map(media = somma/validi, tasso = cancellati/voli, mesi ordinati uniti da "|")
             .sortBy((compagnia, -voli)) ; salva con intestazione in un'unica partizione
```

Minimo e massimo ignorano i voli senza ritardo: un valore fittizio (per esempio 0) falserebbe il massimo di una coppia con soli anticipi.

Spark SQL. Una sola query su una vista temporanea del DataFrame (letto con inferenza dello schema):

```
SELECT op_unique_carrier AS compagnia, origin AS aeroporto_partenza, COUNT(*) AS numero_voli,
       MIN(arr_delay), MAX(arr_delay), ROUND(AVG(arr_delay), 2), ROUND(AVG(cancelled), 4),
       ARRAY_JOIN(SORT_ARRAY(COLLECT_SET(month)), '|') AS mesi_operativi
FROM flights_data GROUP BY op_unique_carrier, origin ORDER BY compagnia, numero_voli DESC
```

Hive. Tabella esterna sulla cartella del dataset e `INSERT OVERWRITE DIRECTORY` con la stessa query, eseguita come 2 job MapReduce (aggregazione, poi ordinamento globale con un reducer). Differenze rispetto a Spark SQL: (i) Hive non applica `skip.header.line.count` in questa configurazione, quindi il filtro `WHERE month IS NOT NULL` esclude le intestazioni (una per file nelle repliche); (ii) `array_join` non esiste in Hive 3 (usato su EMR), perciò i mesi sono ordinati come stringhe a due cifre e privati dello zero iniziale con `regexp_replace`; (iii) ritardi dichiarati `DOUBLE` (con `FLOAT` le medie differirebbero da Spark) e `NULL DEFINED AS ''` per scrivere i mancanti come Spark.

Comp.	Aerop.	Voli	Rit. min	Rit. max	Rit. medio	Tasso canc.	Mesi
9E	LGA	32011	-69.0	1100.0	-3.05	0.035	1|2|3|4|5|6|7|8|9|10|11|12
9E	ATL	26798	-43.0	1207.0	2.46	0.0109	1|2|3|4|5|6|7|8|9|10|11|12
9E	JFK	17108	-60.0	1082.0	1.1	0.0345	1|2|3|4|5|6|7|8|9|10|11|12
9E	DTW	12712	-46.0	941.0	0.18	0.0093	1|2|3|4|5|6|7|8|9|10|11|12
9E	MSP	7540	-50.0	893.0	3.14	0.0081	1|2|3|4|5|6|7|8|9|10|11|12
9E	CVG	6605	-56.0	1035.0	2.07	0.0174	1|2|3|4|5|6|7|8|9|10|11|12
9E	RDU	6425	-47.0	1018.0	8.06	0.0297	1|2|3|4|5|6|7|8|9|10|11|12
9E	CLT	3133	-53.0	1027.0	9.04	0.0393	1|2|3|4|5|6|7|8|9|10|11|12
9E	TYS	2909	-49.0	781.0	-4.82	0.0172	1|2|3|4|5|6|7|8|9|10|11|12
9E	ROC	2683	-49.0	1184.0	1.37	0.0335	1|2|3|4|5|6|7|8|9|10|11|12

Tabella 3: Job 1: prime 10 righe sul dataset completo (1.738 righe in tutto), identiche per le tre tecnologie. Ritardi in minuti.

6 Job 2 — Report dei ritardi per aeroporto e mese

Per ogni coppia (aeroporto di partenza, mese): voli nelle fasce di ritardo in partenza — basso (< 15 minuti, anticipi compresi), medio (da 15 a 60 inclusi), alto (> 60) —, ritardo medio in partenza e in arrivo per fascia e le tre cause di cancellazione o ritardo più frequenti. Le cause sono i codici `CANC_*` e `DELAY_*` (Sezione 2), ordinate per frequenza e, a parità, alfabeticamente: senza spareggio l'ordine dipenderebbe dall'esecuzione. Senza cause il campo vale `N/D`.

Spark Core. Due pipeline sullo stesso RDD, messo in cache perché letto due volte, e un join finale:

```
dati ← textFile(input) senza intestazione ; dati.cache()
// per ogni fascia: (voli, somma dep, voli con arr, somma arr) → vettore di 12 valori
fasce ← dati.map(r → ((r.origine, r.mese), vettore con i valori nella fascia di r)).reduceByKey(somma)
           .mapValues(voli e medie per fascia)
cause ← dati.flatMap(r → [((origine, mese, codice), 1) per il codice di cancellazione e di ritardo, se presenti
    ])
           .reduceByKey(+).map(((o, m, c), n) → ((o, m), (c, n))).groupByKey()
           .mapValues(ordina per (-n, c), prime 3 unite da "|")
output ← fasce.leftOuterJoin(cause) (mancanti → "N/D").sortBy((aeroporto, mese))
```

Il conteggio separato dei voli con ritardo in arrivo rende le medie identiche a SQL; il `groupByKey` è innocuo perché dopo il `reduceByKey` ogni gruppo ha al più nove elementi.

Spark SQL e Hive. La stessa query nei due sistemi: `CASE WHEN` per le fasce, `UNION ALL` dei codici di cancellazione e di ritardo, conteggio, classifica con `ROW_NUMBER() OVER (PARTITION BY aeroporto, mese ORDER BY frequenza DESC, causa)` e pivot per posizione (`CONCAT_WS('|', MAX(CASE WHEN ranking = 1 THEN causa END), …)`) al posto di `COLLECT_LIST`, che non garantisce l'ordine; infine un `LEFT JOIN` con le fasce. Hive esegue il piano come 6 job MapReduce in sequenza, materializzando su HDFS ogni risultato intermedio; Spark esegue lo stesso piano in memoria.

Aerop.	Mese	B: voli	B: dep	B: arr	M: voli	M: dep	M: arr	A: voli	A: dep	A: arr	Top 3 cause
ABE	1	275	-5.51	-14.56	30	34.9	32.5	30	241.3	234.03	LATE, NAS, CAR
ABE	2	296	-6.62	-19.57	19	28.53	10.11	14	227.93	217.93	CAR, NAS, LATE
ABE	3	339	-6.19	-18.37	28	30.68	19.89	23	173.43	163.39	NAS, LATE, CAR
ABE	4	303	-6.45	-17.3	29	33.45	27.66	29	258.66	250.38	CAR, LATE, NAS
ABE	5	320	-6.4	-12.5	26	30.08	29.58	39	183.13	177.13	LATE, NAS, CAR
ABE	6	335	-6.85	-16.46	37	26.65	17.22	37	173.24	172.32	CAR, LATE, NAS
ABE	7	280	-6.36	-10.59	41	34.61	30.15	37	138.08	137.08	LATE, CAR, NAS
ABE	8	289	-6.64	-14.06	42	32.02	27.98	27	146.96	144.3	LATE, CAR, NAS
ABE	9	276	-6.86	-15.9	24	38.08	32.88	16	265.06	261.25	LATE, CAR, WEA
ABE	10	302	-6.03	-17.9	31	26.23	11.97	18	193.17	183.83	LATE, C:WEA, CAR

Tabella 4: Job 2: prime 10 righe sul dataset completo (4.039 righe), identiche per le tre tecnologie. B, M, A: fasce basso, medio, alto; "dep" e "arr": ritardi medi in minuti. Cause: LATE aeromobile in ritardo, CAR compagnia, NAS sistema aereo nazionale, WEA meteo, SEC sicurezza; il prefisso C: indica una cancellazione (`CANC_*`), altrimenti è una causa di ritardo (`DELAY_*`).

Verifica della correttezza

Lo script `verify_outputs.py` confronta gli output delle tre tecnologie riga per riga (valori numerici con tolleranza pari all'arrotondamento, stringhe in modo esatto) ed è eseguito dopo ogni gruppo di esecuzioni sullo stesso dataset. Le tre tecnologie producono lo stesso risultato su tutti i dataset, sia in locale sia su YARN. Durante lo sviluppo la verifica ha fatto emergere l'intestazione contata come volo da Hive, il massimo falsato in Spark Core, la collisione dei codici delle cause e l'ordine non deterministico delle cause a pari frequenza.

7 Risultati sperimentali

Le Tabelle 5–7 riportano il tempo totale in secondi; la Figura 1 li confronta per tecnologia e job.

Job	Tecnologia	1%	20%	50%	70%	100%	2×	5×
Job 1	Spark Core	6,8	8,8	10,8	12,2	14,6	18,9	32,2
	Spark SQL	7,3	10,1	11,7	13,6	14,5	23,2	60,1
	Hive	12,5	13,4	15,2	17,5	19,5	27,6	52,7
Job 2	Spark Core	6,9	9,0	13,0	16,2	20,8	28,9	63,9
	Spark SQL	8,7	10,0	12,5	14,7	17,1	25,3	58,7
	Hive	21,3	22,9	25,3	26,9	30,1	40,5	70,1

Tabella 5: Tempo totale (s) in locale.

Job	Tecnologia	1%	20%	50%	70%	100%	2×	5×
Job 1	Spark Core	18,9	19,5	22,0	23,7	29,7	34,7	64,5
	Spark SQL	20,5	24,3	27,5	31,6	37,3	54,1	106,8
	Hive	47,7	50,0	48,8	49,4	53,9	54,0	72,2
Job 2	Spark Core	18,4	29,2	26,1	31,3	33,5	51,9	114,9
	Spark SQL	20,2	25,7	30,8	34,2	39,1	60,2	124,6
	Hive	130,8	139,1	134,4	133,2	138,6	140,2	175,1

Tabella 6: Tempo totale (s) sullo pseudo-cluster YARN.

Job	Tecnologia	1%	20%	50%	70%	100%
Job 1	Spark Core	31,9	35,3	39,1	46,8	48,3
	Spark SQL	36,4	41,9	48,7	48,0	52,4
	Hive	35,8	32,7	40,2	40,6	43,1
Job 2	Spark Core	28,0	36,4	41,1	49,7	50,8
	Spark SQL	37,9	44,9	50,5	53,0	59,1
	Hive	38,4	42,6	42,7	48,5	54,0

Tabella 7: Tempo totale (s) su AWS EMR (1 primario + 2 core `m5.xlarge`).

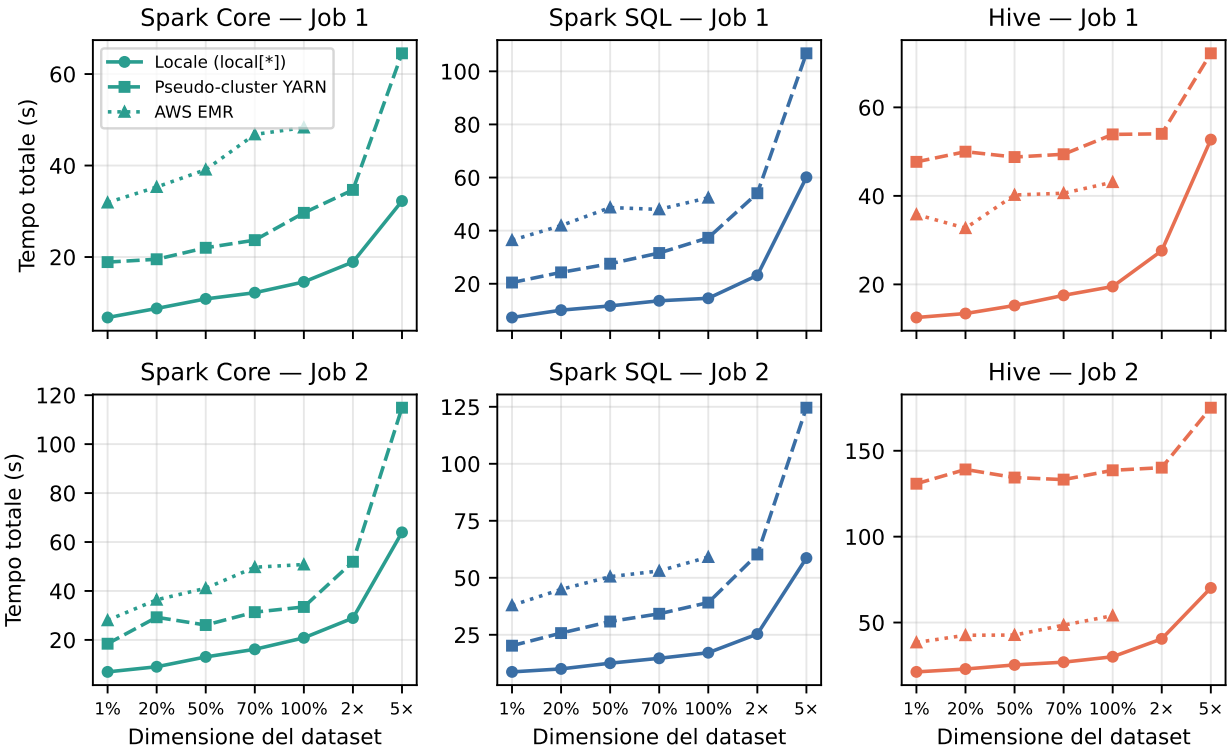


Figura 1: Tempo totale al variare della dimensione del dataset nei tre ambienti (su EMR dall'1% al 100%).

Job	Tecnologia	Locale	YARN	AWS EMR	YARN/Locale	AWS/Locale
Job 1	Spark Core	14,6	29,7	48,3	2,04×	3,32×
	Spark SQL	14,5	37,3	52,4	2,57×	3,61×
	Hive	19,5	53,9	43,1	2,76×	2,21×
Job 2	Spark Core	20,8	33,5	50,8	1,61×	2,44×
	Spark SQL	17,1	39,1	59,1	2,28×	3,45×
	Hive	30,1	138,6	54,0	4,61×	1,80×

Tabella 8: Tempo totale (s) sul dataset completo nei tre ambienti e rapporto rispetto al locale.

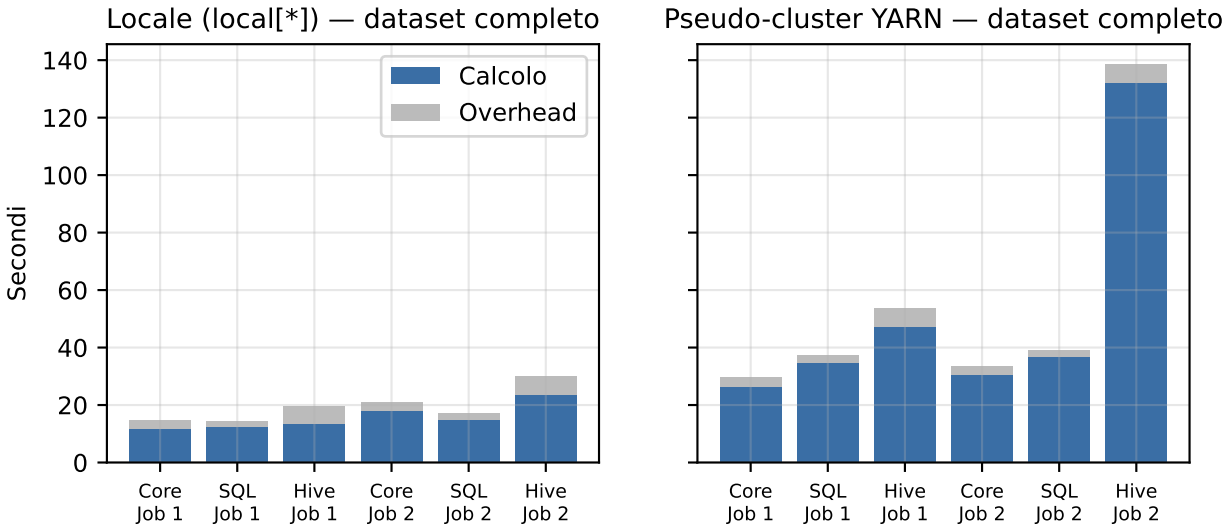


Figura 2: Tempo di calcolo e overhead sul dataset completo in locale e su YARN.

Dataset	Job	Tecnologia	Job lanciati	Stage	Task	MB scambiati
100%	Job 1	Spark Core	5 job	7	11	0,2 (shuffle)
100%	Job 1	Spark SQL	8 job	8	11	0,4 (shuffle)
100%	Job 1	Hive	2 MR	–	–	206 (HDFS)
100%	Job 2	Spark Core	5 job	10	25	1,4 (shuffle)
100%	Job 2	Spark SQL	14 job	14	23	2,7 (shuffle)
100%	Job 2	Hive	6 MR	–	–	413 (HDFS)
5×	Job 1	Spark Core	5 job	7	43	0,6 (shuffle)
5×	Job 1	Spark SQL	8 job	8	35	1,6 (shuffle)
5×	Job 1	Hive	2 MR	–	–	1.028 (HDFS)
5×	Job 2	Spark Core	5 job	10	113	4,5 (shuffle)
5×	Job 2	Spark SQL	14 job	14	84	10,9 (shuffle)
5×	Job 2	Hive	6 MR	–	–	2.057 (HDFS)

Tabella 9: Metriche del motore su YARN: job, stage e task di Spark con i MB scritti nello shuffle; job MapReduce di Hive con i MB letti da HDFS.

8 Analisi critica

8.1 Espressività

Spark SQL e Hive esprimono ciascun job con una sola query dichiarativa: raggruppamenti, espressioni condizionali per le fasce, funzioni finestra per la classifica delle cause e join esterno sono costrutti

nativi, e il piano di esecuzione è scelto dal sistema. In Spark Core la stessa logica va costruita a mano: accumulatori per le aggregazioni, gestione esplicita dei valori mancanti, due pipeline e un join per il Job 2, ordinamento e intestazione dell'output. Le righe di codice significative (Tabella 10) lo riflettono solo in parte, perché gli script SQL includono anche la definizione della tabella e le colonne di output; la differenza vera è che in Spark Core ogni riga è logica da scrivere e verificare. Spark Core è però il più flessibile: qualunque struttura (per esempio un insieme di mesi) può fare da accumulatore, senza dipendere dalle funzioni disponibili. Hive ha il dialetto più limitato, e la portabilità tra versioni non è garantita: `array_join` esiste in Hive 4 ma non in Hive 3.

Tecnologia	Job 1	Job 2
Spark Core	79	105
Spark SQL	48	75
Hive	35	83

Tabella 10: Righe di codice significative per job (escluse righe vuote, commenti e stampe di servizio).

8.2 Semplicità implementativa

Spark SQL è stato il più semplice: la query si scrive e si verifica come in un database. Spark Core ha richiesto più attenzione ai dettagli semantici che SQL gestisce da solo: tre dei difetti emersi dalla verifica (massimo falsato dai voli senza ritardo, medie in arrivo con valori mancanti contati come zero, ordine delle cause a pari frequenza) riguardavano la versione RDD o la sua coerenza con SQL. Hive, a parità di query, è il più oneroso da mettere in esercizio: metastore, compatibilità con Java 21, intestazioni non ignorate, tipi (`FLOAT` contro `DOUBLE`) e formato dei valori mancanti hanno richiesto interventi specifici.

8.3 Efficienza

- **Dataset piccoli**: il tempo è dominato dai costi fissi. In locale sull'1% Spark impiega circa 6,8–8,7 s, di cui il 48 % (Spark Core, Job 1) è overhead di avvio; Hive paga l'avvio di un job MapReduce per ogni fase (12,5 s per il Job 1 e 21,3 s per il Job 2, che ne lancia 6).
- **Dataset grandi, in locale**: sul 5× Spark Core è il più veloce nel Job 1 (32,2 s contro 60,1 s di Spark SQL e 52,7 s di Hive), mentre nel Job 2 le tre tecnologie sono vicine (63,9, 58,7 e 70,1 s). Il divario di Spark SQL nel Job 1 dipende dal numero di letture dell'input: l'event log mostra 3 letture complete da 1 GB (inferenza dello schema, 7,9 s; anteprima con `show`, 26,8 s; scrittura, 18,1 s), perché ogni azione su un DataFrame non in cache riesegue la query. Spark Core legge l'input una sola volta: le azioni successive riusano i file di shuffle già prodotti e Spark salta gli stage già calcolati.
- **YARN**: lo stesso hardware diventa più lento (Tabella 8): sul dataset completo da 1,61 a 2,57 volte per Spark e fino a 4,61 volte per il Job 2 di Hive. Il costo è in gran parte dentro il tempo di calcolo (Figura 2): richiesta dei container a YARN, avvio delle JVM degli executor e, per Spark, solo 2 executor da un core contro gli 8 thread del locale. Hive lo paga a ogni job MapReduce: il Job 2 resta tra 130,8 e 175,1 s a ogni dimensione.
- **AWS EMR**: con questi volumi il cluster è più lento del locale (da 1,80 a 3,61 volte sul dataset completo): tre nodi, rete e coordinamento non vengono ripagati da appena 205,5 MB. Hive, che su EMR usa Tez (un unico grafo di esecuzione invece di job MapReduce separati), è il più veloce nel Job 1 (43,1 s contro 48,3 e 52,4 s).

8.4 Scalabilità

Moltiplicando l'input per 100 (dall'1% al 100%) i tempi crescono molto meno che linearmente: in locale di 1,4–3,0 volte, su EMR di 1,2–1,8 volte e su YARN di appena 1,1–1,9 volte, perché fino al dataset completo prevalgono i costi fissi. Oltre, la crescita si avvicina a quella dei dati: da 100% a 5× (input ×5) in locale Spark Core cresce di 2,2–3,1 volte e Spark SQL di 3,4–4,1, penalizzato dalle letture ripetute. Hive in locale cresce in modo simile a Spark Core (2,3–2,7 volte), mentre su YARN cresce molto meno (circa 1,3 volte, contro 2,2–3,4 di Spark Core) perché lì il suo costo fisso è molto più alto: al 5× Hive supera Spark SQL nel Job 1 (72,2 contro 106,8 s). Le curve di Spark e Hive tendono quindi a incrociarsi al crescere dell'input: il vantaggio dell'esecuzione in memoria è netto sui volumi piccoli e medi e si riduce quando domina la lettura dei dati. La dimensione del cluster EMR è rimasta fissa; con questi volumi più nodi aumenterebbero soprattutto il coordinamento.

8.5 Shuffle, aggregazione e preparazione dei dati

- **Shuffle**: i dati scambiati sono pochi MB anche con 1 GB di input (Tabella 9): 0,6 MB per Spark Core e 10,9 MB per Spark SQL nel Job 2 sul 5×. Entrambi aggregano *prima* dello shuffle (`reduceByKey` combina in ogni partizione, Catalyst usa un'aggregazione hash parziale), e le chiavi sono poche (circa 1.700 coppie compagnia-aeroporto, 4.000 aeroporto-mese). Il costo dominante è quindi leggere e interpretare il CSV, non scambiare dati.
- **Materializzazione in Hive**: ogni job MapReduce scrive il proprio risultato su HDFS e il successivo lo rilegge; inoltre la query del Job 2 legge l'input due volte (fasce e cause), come mostrano i 413 MB letti per 205,5 MB di input.
- **Valutazione pigra**: in Spark SQL il Job 2 legge l'input 5 volte (inferenza dello schema, poi due rami della query, fasce e cause, sia per l'anteprima sia per la scrittura). Uno schema esplicito e la cache del risultato prima delle due azioni eliminerebbero la maggior parte di queste letture. In Spark Core la cache dell'input del Job 2 evita solo in parte la doppia lettura, perché le due pipeline sono eseguite in stage paralleli (19,4 e 53,9 s sul 5×).
- **Preparazione**: ridurre il file da 35 a 9 colonne (da 1,2 GB a 205,5 MB) riduce di circa sei volte il volume letto da ogni esecuzione di ogni job, che è proprio la componente dominante. Anche le altre scelte hanno effetto sulle prestazioni: i codici espliciti permettono di contare insieme cancellazioni e ritardi con un solo `UNION ALL`, e i dataset come cartelle HDFS evitano copie preliminari.

9 Conclusioni

Le tre tecnologie producono risultati identici. Spark SQL offre il miglior rapporto tra espressività e semplicità ed è competitivo sui volumi piccoli e medi, a patto di controllare le letture ripetute dovute alla valutazione pigra e all'inferenza dello schema. Spark Core è il più veloce quando la pipeline è semplice, perché legge l'input una volta e riusa i risultati intermedi, ma richiede molto più codice e attenzione ai dettagli semantici. Hive ha il costo fisso più alto, per l'avvio dei job MapReduce e la materializzazione dei risultati intermedi, ma scala in modo più regolare al crescere dell'input ed è competitivo su EMR con Tez. Sui volumi di questo dataset il fattore dominante è l'overhead di avvio e coordinamento, più che la capacità di calcolo: per questo il locale batte YARN ed EMR, e la preparazione dei dati, che riduce il volume letto, conta più delle differenze negli shuffle.

A Riproducibilità

Il repository (`https://github.com/fragiovis/BigData-FlightAnalysis`) contiene un modulo per tecnologia (`spark-core/`, `spark-sql/`, `hive/`, ciascuno con `run.sh` per l'ambiente locale e

`run_aws.sh` per EMR), la preparazione dei dati (`dataset/`), l'esecuzione e la misura dei job (`bench/`, `benchmark.py`), la verifica (`verify_outputs.py`), una dashboard Streamlit per lanciare i job e consultarne tempi, stage, log e risultati (`dashboard/`), i risultati (`results/`) e i sorgenti di questa relazione. Il README descrive installazione e configurazione; i passi essenziali sono:

```
pip install -r requirements.txt && cd dataset && python3 download.py   # dataset (token Kaggle)
bash generate_data.sh "local[*]" && cd ..                              # preparazione, porzioni, repliche, HDFS
python3 benchmark.py job_1 yarn && python3 benchmark.py job_2 yarn     # anche "local[*]" o dalla dashboard
python3 verify_outputs.py job_1 && python3 verify_outputs.py job_2
python3 docs/relazione/genera_dati.py && cd docs/relazione && tectonic main.tex
```